

UNIVERSAL PROJECT PLAYBOOK

# Lovable-Quality AI Coding Workflows

A reusable prompt, context, execution, and verification system for any software project

Replace every {{PLACEHOLDER}} with project-specific information.  
Use the full system, or copy only the modules you need.

**Edition 1.0 | August 2026**

# How to use this guide

## The promise

This playbook helps a general coding agent behave more like a specialized product builder: it receives better context, follows a deliberate workflow, verifies its work, and applies consistent quality standards. It does not reproduce any private Lovable prompt.

## The 15-minute setup

1. Complete the Project Profile in Section 3.
2. Add the Repository Instructions from Section 4 to AGENTS.md, CLAUDE.md, or your agent rules file.
3. Start each substantial task with the Master Task Prompt in Section 5.
4. Attach the relevant Domain Pack and Task Pack.
5. Require the Verification Gate before accepting completion.
6. Use the Quality Scorecard to review the result and improve your rules after recurring mistakes.

## The operating model

|                                                                                                                        |                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Context</b><br>Give the agent product goals, architecture, constraints, current state, and proof sources.        | <b>2. Plan</b><br>Require repository exploration, assumptions, risks, acceptance criteria, and a coherent implementation sequence. |
| <b>3. Build</b><br>Make the smallest complete change while preserving unrelated behavior and existing conventions.     | <b>4. Verify</b><br>Run automated checks, inspect runtime behavior, review the diff, and test the real user flow.                  |
| <b>5. Refine</b><br>Correct visual, performance, security, and edge-case defects instead of stopping at “it compiles.” | <b>6. Learn</b><br>Convert recurring failures into permanent repository instructions and regression tests.                         |

## Important

A long prompt cannot compensate for a broken environment. The agent needs working commands, accessible logs, representative test data, and tools that can observe the result.

## What “Lovable-like quality” means here

- **Clear intent.** The agent understands who the user is, what outcome matters, and how success will be judged.
- **Opinionated defaults.** The project has explicit design, architecture, testing, and safety conventions.
- **Repository awareness.** The agent reads relevant code and uses existing abstractions before creating new ones.
- **Visible verification.** Completion requires evidence, not a confident paragraph.
- **End-to-end coherence.** UI, API, data, error states, security, and documentation change together when required.
- **Controlled scope.** The agent does not redesign or refactor unrelated areas merely because it can.

# Contents

---

- 01 [Quality System](#)
- 02 [Placeholder Language](#)
- 03 [Project Profile](#)
- 04 [Repository Instructions](#)
- 05 [Master Task Prompt](#)
- 06 [The Five-Phase Agent Loop](#)
- 07 [Universal Quality Guardrails](#)
- 08 [Domain Packs](#)
- 09 [Task Packs](#)
- 10 [Verification and Evidence](#)
- 11 [Recovery and Debugging](#)
- 12 [Quality Scorecard](#)
- 13 [Ready-to-Paste Complete Kit](#)
- 14 [Worked Example](#)
- 15 [One-Page Checklists](#)

## Best practice

Keep the stable project rules in a repository file. Keep task-specific details in each request. This prevents enormous repetitive prompts while preserving consistent behavior.

# The Quality System

Why the same model can produce dramatically different results.

The goal is not to make the model “more intelligent.” The goal is to remove avoidable uncertainty and force a reliable feedback loop. A high-quality coding agent is a system, not a single prompt.

## The quality equation

```
RESULT QUALITY =
  model capability
+ relevant context
+ explicit constraints
+ useful tools
+ staged workflow
+ observable verification
+ strong product defaults
- ambiguity
- unnecessary scope
- untested assumptions
```

*Treat this as a diagnostic model. When results disappoint, identify the weak term instead of simply making the prompt longer.*

## Seven layers of a strong agent harness

|                                                                                                                      |                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Product layer</b><br>User, outcome, business rules, brand, and product principles.                             | <b>2. Architecture layer</b><br>Components, boundaries, data flow, dependencies, conventions, and technical constraints.      |
| <b>3. Context layer</b><br>Relevant files, schemas, logs, screenshots, tickets, examples, and current runtime state. | <b>4. Workflow layer</b><br>Explore, plan, implement, verify, refine, and report.                                             |
| <b>5. Tool layer</b><br>Repository search, shell, tests, browser, database, observability, and deployment tools.     | <b>6. Quality layer</b><br>UX, correctness, security, accessibility, performance, maintainability, and operational readiness. |
| <b>7. Learning layer</b><br>Regression tests and durable instructions created from repeated failures.                |                                                                                                                               |

## Prompt size versus prompt value

- Prefer precise constraints over motivational language such as “be brilliant” or “make it amazing.”
- Provide examples where taste or format matters.
- Reference sources of truth instead of restating code or schemas from memory.
- Separate permanent project rules from temporary task requirements.
- Ask for evidence from tools. Do not ask the model to “double-check” only in its head.

- Keep one task focused enough to verify end to end.

### Anti-pattern

"Build a production-ready, beautiful, scalable app with perfect UX" contains ambition but almost no executable information.

## Minimum environment capabilities

- [ ] A deterministic install command and documented runtime version.
- [ ] A development or preview command the agent can start.
- [ ] Typecheck, lint, unit test, integration test, and build commands where applicable.
- [ ] Access to browser or device inspection for user-facing work.
- [ ] Representative environment variables or a safe local substitute.
- [ ] Seed data, fixtures, mocks, or a test account for real flows.
- [ ] Logs and error output that are visible to the agent.
- [ ] A clean way to inspect changed files and revert accidental edits.

# Placeholder Language

A consistent find-and-replace vocabulary for every template.

All placeholders use double braces. Replace them before sending a prompt, or leave a placeholder only when the agent can discover its value from the repository. Do not leave critical business or safety decisions for the agent to invent.

| Placeholder               | What to enter                              | Example                                       |
|---------------------------|--------------------------------------------|-----------------------------------------------|
| {{PROJECT_NAME}}          | Name of the product or repository          | Northstar Console                             |
| {{PROJECT_TYPE}}          | Web, mobile, API, library, ML, infra, etc. | B2B full-stack web app                        |
| {{TARGET_USER}}           | Primary user or operator                   | Support team lead                             |
| {{USER_OUTCOME}}          | Result the user wants                      | Resolve an escalated case quickly             |
| {{BUSINESS_GOAL}}         | Business value                             | Reduce average resolution time                |
| {{PRIMARY_ACTION}}        | Most important user action                 | Assign and resolve a case                     |
| {{TASK}}                  | Requested change                           | Add bulk case assignment                      |
| {{CURRENT_BEHAVIOR}}      | What happens now                           | Cases must be assigned one by one             |
| {{DESIRED_BEHAVIOR}}      | What should happen                         | Select many cases and assign once             |
| {{ACCEPTANCE_CRITERIA}}   | Observable completion rules                | Selections persist; invalid cases are skipped |
| {{NON_GOALS}}             | Explicitly excluded work                   | No redesign of the case table                 |
| {{TECH_STACK}}            | Frameworks, languages, infrastructure      | Next.js, TypeScript, Postgres                 |
| {{ARCHITECTURE}}          | System boundaries and data flow            | Web client -> API -> database                 |
| {{SOURCE_OF_TRUTH}}       | Files, schemas, designs, tickets           | /docs/spec.md and schema.prisma               |
| {{DESIGN_DIRECTION}}      | Visual and interaction character           | Quiet, dense, enterprise utility              |
| {{CONSTRAINTS}}           | Hard limits                                | No new dependency; preserve API compatibility |
| {{SECURITY_REQUIREMENTS}} | Auth, privacy, compliance needs            | Tenant isolation and audit logging            |
| {{PERFORMANCE_TARGET}}    | Latency, size, throughput target           | P95 API response under 300 ms                 |
| {{TEST_COMMANDS}}         | Commands that prove correctness            | npm test; npm run typecheck                   |

| Placeholder                        | What to enter                       | Example                                 |
|------------------------------------|-------------------------------------|-----------------------------------------|
| <code>{{RUN_COMMAND}}</code>       | Command to run the product          | npm run dev                             |
| <code>{{DEPLOYMENT_TARGET}}</code> | Runtime or platform                 | Kubernetes staging cluster              |
| <code>{{EVIDENCE_REQUIRED}}</code> | Proof expected at completion        | Test output, screenshots, changed files |
| <code>{{KNOWN_RISKS}}</code>       | Expected regressions or uncertainty | Permission edge cases                   |
| <code>{{OPEN_QUESTIONS}}</code>    | Unresolved product decisions        | Should archived cases be selectable?    |

## Rules for placeholders

- Replace a placeholder with concrete, testable language whenever possible.
- Use “discover from repository” only for facts the agent can reliably inspect.
- Use “not applicable” rather than silently deleting a safety or verification category.
- Keep secrets out of prompts. Refer to secret names or secure stores, never values.
- When a decision is genuinely unresolved, list the available options and the decision owner.

### Fast replacement header

```
PROJECT: {{PROJECT_NAME}}
TYPE: {{PROJECT_TYPE}}
USER: {{TARGET_USER}}
TASK: {{TASK}}
OUTCOME: {{USER_OUTCOME}}
STACK: {{TECH_STACK}}
CONSTRAINTS: {{CONSTRAINTS}}
ACCEPTANCE: {{ACCEPTANCE_CRITERIA}}
NON-GOALS: {{NON_GOALS}}
PROOF: {{EVIDENCE_REQUIRED}}
```

*Paste this above a task when the full Project Profile already lives in the repository.*

# Project Profile

Create a durable product and engineering memory for the agent.

Complete this once and keep it in the repository as PROJECT\_CONTEXT.md, docs/agent-context.md, or equivalent. Update it when architecture, product strategy, or operating constraints change.

## Universal Project Profile

```
# {{PROJECT_NAME}} - Project Profile

## Product
- Project type: {{PROJECT_TYPE}}
- Primary users: {{TARGET_USER}}
- Core user outcome: {{USER_OUTCOME}}
- Business goal: {{BUSINESS_GOAL}}
- Primary action: {{PRIMARY_ACTION}}
- Product principles: {{PRODUCT_PRINCIPLES}}
- Non-negotiable business rules: {{BUSINESS_RULES}}

## Experience
- Design direction: {{DESIGN_DIRECTION}}
- Accessibility target: {{ACCESSIBILITY_TARGET}}
- Supported devices/platforms: {{SUPPORTED_PLATFORMS}}
- Critical user journeys: {{CRITICAL_JOURNEYS}}
- Required states: loading, empty, error, partial, success, offline where relevant

## Technology
- Stack: {{TECH_STACK}}
- Architecture: {{ARCHITECTURE}}
- Key directories: {{KEY_DIRECTORIES}}
- Data stores and schemas: {{DATA_SOURCES}}
- External services: {{INTEGRATIONS}}
- Runtime/deployment: {{DEPLOYMENT_TARGET}}

## Engineering rules
- Existing conventions: {{CONVENTIONS}}
- Dependency policy: {{DEPENDENCY_POLICY}}
- Compatibility requirements: {{COMPATIBILITY_REQUIREMENTS}}
- Performance targets: {{PERFORMANCE_TARGET}}
- Security requirements: {{SECURITY_REQUIREMENTS}}
- Privacy/compliance: {{PRIVACY_REQUIREMENTS}}

## Commands
- Install: {{INSTALL_COMMAND}}
- Run: {{RUN_COMMAND}}
- Typecheck: {{TYPECHECK_COMMAND}}
- Lint: {{LINT_COMMAND}}
- Unit tests: {{UNIT_TEST_COMMAND}}
- Integration tests: {{INTEGRATION_TEST_COMMAND}}
- End-to-end tests: {{E2E_COMMAND}}
- Production build: {{BUILD_COMMAND}}

## Sources of truth
- Product specification: {{PRODUCT_SPEC_PATH}}
- Designs: {{DESIGN_SOURCE}}
- API contract: {{API_CONTRACT_PATH}}
- Database schema: {{SCHEMA_PATH}}
- Operational runbook: {{RUNBOOK_PATH}}
- Known issues: {{KNOWN_ISSUES_PATH}}

## Definition of done
```



{{GLOBAL\_DEFINITION\_OF\_DONE}}

*Keep this factual. Avoid aspirational filler that cannot guide a concrete implementation decision.*

## Global definition of done template

- [ ] The requested behavior works through the real user or system flow.
- [ ] Relevant tests pass and new behavior has appropriate coverage.
- [ ] No unrelated behavior, public API, schema, or visual language is changed.
- [ ] Error, empty, loading, permission, and failure states are handled where relevant.
- [ ] Security, privacy, accessibility, and performance requirements are met.
- [ ] The production build succeeds and runtime logs contain no new unexplained errors.
- [ ] Changed behavior is documented for users, operators, or developers as appropriate.
- [ ] The final report includes changed files, commands run, evidence, assumptions, and residual risks.

### Keep it fresh

An inaccurate context file is worse than a short one. Assign ownership and review it when architecture or business rules change.

# Repository Instructions

The durable agent operating system for AGENTS.md or equivalent.

This is the most important reusable artifact. Store it at the repository root, then add narrower instruction files inside subdirectories only when those areas need different rules.

## Universal AGENTS.md

```
# Agent operating rules for {{PROJECT_NAME}}
```

## Mission

Build and maintain {{PROJECT\_TYPE}} software that helps {{TARGET\_USER}} achieve {{USER\_OUTCOME}}. Optimize for correctness, coherent user/system flows, maintainability, security, and observable verification.

## Before editing

1. Read the nearest instruction files and relevant project documentation.
2. Inspect the relevant code, tests, types, schema, and current behavior.
3. Search for existing components, utilities, patterns, and similar features.
4. Identify acceptance criteria, non-goals, risks, and required proof.
5. For non-trivial work, state a concise plan before implementation.

## Implementation rules

- Make the smallest coherent change that fully solves the task.
- Reuse existing abstractions before creating new ones.
- Follow existing naming, architecture, error handling, and styling conventions.
- Preserve unrelated behavior and user-authored changes.
- Do not add a dependency unless it materially improves the solution and the existing stack cannot reasonably provide the capability.
- Do not change a public API, database schema, authentication behavior, permissions, or deployment configuration silently.
- Keep business rules in testable domain logic rather than scattered UI code.
- Handle relevant loading, empty, error, partial, permission, timeout, retry, and success states.
- Never expose secrets, personal data, internal tokens, or privileged errors.

## Quality rules

- Correctness: validate inputs, invariants, edge cases, and failure paths.
- UX: preserve hierarchy, responsiveness, accessibility, and clear feedback.
- Security: enforce authorization server-side and validate trust boundaries.
- Performance: avoid unnecessary work, repeated queries, oversized payloads, blocking operations, and unbounded loops or collections.
- Maintainability: prefer clear boundaries, descriptive names, and local reasoning over cleverness.
- Operations: add useful logs, metrics, migrations, rollback notes, or runbook updates when the change affects production behavior.

## Verification gate

Before declaring completion:

1. Review the diff for accidental or unrelated changes.
2. Run the most relevant typecheck, lint, tests, and build commands.
3. Run the affected product or service and exercise the real flow.
4. Inspect browser/device output, logs, network calls, data changes, or API responses as appropriate.
5. Check representative edge cases and at least one failure path.
6. Fix failures or clearly report blockers with evidence.
7. Report changed files, commands and results, assumptions, and remaining risk.

## Communication

- Be concise and concrete.
- Distinguish observed facts from assumptions.

- Ask only when a missing product decision would materially change the result; otherwise state a reasonable assumption and proceed.
- Never claim a check passed unless it was actually run.
- Never describe incomplete work as complete.

## Project-specific commands and references  
{{PROJECT\_COMMANDS\_AND\_REFERENCES}}

*Adjust only the project-specific final section unless a rule conflicts with your team's engineering policy.*

## Directory-level addendum

### Nested instruction file

# Rules for {{DIRECTORY\_OR\_COMPONENT}}

- Purpose: {{AREA\_PURPOSE}}
- Owners: {{OWNERS}}
- Local architecture: {{LOCAL\_ARCHITECTURE}}
- Required patterns: {{REQUIRED\_PATTERNS}}
- Forbidden patterns: {{FORBIDDEN\_PATTERNS}}
- Local test command: {{LOCAL\_TEST\_COMMAND}}
- Compatibility constraints: {{LOCAL\_COMPATIBILITY}}
- Review checklist: {{LOCAL\_REVIEW\_CHECKLIST}}

*Use this for migrations, mobile code, infrastructure, generated code, security boundaries, or other specialized areas.*

## Instruction hygiene

- Keep rules short enough to remain visible in the agent context.
- Turn recurring mistakes into explicit rules or tests.
- Delete obsolete instructions promptly.
- Do not duplicate the same rule at many levels unless the tool requires it.
- Use examples for formatting, API style, design language, or migration conventions that are easy to misunderstand.

# Master Task Prompt

A universal request template for substantial changes.

Use this for features, integrations, refactors, migrations, production bugs, and any task where local correctness is not enough. Remove sections that truly do not apply, but keep the workflow and verification gate.

## Master Task Prompt

You are working in `{{PROJECT_NAME}}`, a `{{PROJECT_TYPE}}` for `{{TARGET_USER}}`.

## Objective  
`{{TASK}}`

## User/system outcome  
`{{USER_OUTCOME}}`

## Current behavior  
`{{CURRENT_BEHAVIOR}}`

## Desired behavior  
`{{DESIRED_BEHAVIOR}}`

## Acceptance criteria  
`{{ACCEPTANCE_CRITERIA}}`

## Non-goals  
`{{NON_GOALS}}`

## Context and sources of truth

- Product/specification: `{{SOURCE_OF_TRUTH}}`
- Relevant areas: `{{RELEVANT_PATHS}}`
- Similar existing implementation: `{{SIMILAR_IMPLEMENTATION}}`
- Data/API contract: `{{DATA_OR_API_CONTRACT}}`
- Designs or examples: `{{DESIGN_OR_EXAMPLES}}`

## Constraints

- Technical: `{{TECHNICAL_CONSTRAINTS}}`
- Compatibility: `{{COMPATIBILITY_REQUIREMENTS}}`
- Security/privacy: `{{SECURITY_REQUIREMENTS}}`
- Performance: `{{PERFORMANCE_TARGET}}`
- Scope: do not modify unrelated behavior or files.

## Required workflow

1. Explore the relevant repository areas before editing.
2. Summarize the current implementation and likely change surface.
3. State assumptions, risks, and a concise implementation plan.
4. Implement the smallest complete solution using existing patterns.
5. Add or update appropriate tests.
6. Run the relevant checks and exercise the real flow.
7. Review the final diff and fix discovered defects.

## Verification

Run or perform:

`{{TEST_COMMANDS}}`  
`{{RUNTIME_VERIFICATION}}`  
`{{EDGE_CASES_TO_VERIFY}}`

## Completion report

Provide:

- concise outcome summary;
- changed files and why;

- checks actually run and results;
- screenshots, logs, responses, or other evidence where relevant;
- assumptions and decisions;
- remaining risks or follow-up work.

Do not claim completion without evidence that the acceptance criteria are met.

*For small tasks, use the compact version below.*

### Compact Task Prompt

Implement {{TASK}} in {{PROJECT\_NAME}}.

Outcome: {{USER\_OUTCOME}}

Acceptance: {{ACCEPTANCE\_CRITERIA}}

Constraints: {{CONSTRAINTS}}

Non-goals: {{NON\_GOALS}}

Relevant sources: {{SOURCE\_OF\_TRUTH}}

First inspect the relevant implementation and existing patterns. Then make the smallest coherent change, add or update tests, run {{TEST\_COMMANDS}}, verify the real affected flow, review the diff, and report evidence plus remaining risks.

*Use only when the repository already contains a strong Project Profile and AGENTS.md.*

## Prompt preflight

- ☐ Can a reviewer observe each acceptance criterion?
- ☐ Are important non-goals explicit?
- ☐ Does the agent know which files or artifacts are authoritative?
- ☐ Are security, privacy, compatibility, and performance constraints stated?
- ☐ Can the agent actually run the requested verification?
- ☐ Is the task small enough to finish and verify coherently?

# The Five-Phase Agent Loop

Separate exploration, planning, implementation, verification, and refinement.

---

A specialized builder feels reliable because it does not treat generation as the end of the job. Use the following phase prompts independently or as a single chained workflow.

## Phase 1 - Explore

### Exploration Prompt

Investigate {{TASK}} without editing files yet.

1. Read the relevant instructions and documentation.
2. Trace the current user/system flow end to end.
3. Identify relevant files, tests, types, schemas, APIs, and runtime behavior.
4. Find existing patterns or similar implementations to reuse.
5. List hidden dependencies, permission boundaries, failure paths, and likely regression areas.
6. Separate observed facts from assumptions.
7. Return a concise change-surface map and unresolved material decisions.

*Use this when the task is ambiguous, cross-cutting, risky, or in an unfamiliar repository.*

## Phase 2 - Plan

### Planning Prompt

Using the exploration findings, produce an implementation plan for {{TASK}}.

The plan must include:

- the intended behavior and acceptance criteria;
- files/components/services likely to change;
- data and control flow;
- reuse versus new abstractions;
- migration or compatibility strategy;
- error, permission, and edge-case behavior;
- tests and runtime verification;
- deployment or rollback considerations;
- risks and assumptions.

Prefer the smallest coherent design. Do not include unrelated cleanup.

*A useful plan makes review easier and prevents premature edits.*

## Phase 3 - Implement

### Implementation Prompt

Implement the approved plan for {{TASK}}.

Rules:

- Preserve unrelated behavior and existing public contracts.
- Reuse existing patterns and design primitives.
- Keep the change set focused and internally consistent.

- Add validation and explicit error handling at trust boundaries.
- Cover relevant non-happy states.
- Add or update tests alongside the implementation.
- Stop and report evidence if a discovered constraint invalidates the plan; otherwise make reasonable local decisions and continue.

*Do not ask the model to "be perfect." Give it concrete invariants and boundaries.*

## Phase 4 - Verify

### Verification Prompt

Verify the implementation for {{TASK}} as a skeptical reviewer.

1. Map every acceptance criterion to direct evidence.
2. Review the diff for accidental scope expansion and hidden regressions.
3. Run the relevant static checks, tests, and production build.
4. Exercise the real affected flow in the running system.
5. Inspect logs, network calls, data changes, screenshots, or outputs.
6. Test representative edge cases, permission boundaries, and one failure path.
7. Check security, accessibility, performance, and compatibility requirements.
8. Fix issues found, then repeat the affected checks.

Report only checks actually performed. Mark any unverified item clearly.

*Verification should use tools and observable output, not only code inspection.*

## Phase 5 - Refine and report

### Final Review Prompt

Perform a final quality pass on {{TASK}}.

Improve only issues directly related to the requested change. Confirm:

- behavior is coherent end to end;
- naming and structure match the repository;
- user/operator feedback is clear;
- loading, empty, error, partial, and success states are appropriate;
- documentation and operational notes are sufficient;
- no debug code, placeholder data, secrets, dead code, or accidental files remain.

Return a concise final report with outcome, changed files, evidence, assumptions, and remaining risk. Do not repeat the full implementation narrative.

*This is the difference between "code was written" and "the change is ready for review."*

# Universal Quality Guardrails

Attach the relevant rules to every project, regardless of stack.

|                                                                                                                                                                                           |                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Correctness</b><br>Define invariants. Validate inputs. Handle concurrency, idempotency, nullability, time, ordering, retries, partial failure, and boundary conditions where relevant. | <b>Scope control</b><br>Change only what the task requires. Avoid broad reformatting, dependency churn, speculative abstractions, and drive-by refactors.          |
| <b>Architecture</b><br>Respect existing boundaries. Keep domain logic separate from presentation and infrastructure. Avoid circular dependencies and hidden global state.                 | <b>Security</b><br>Authenticate and authorize at trusted boundaries. Validate untrusted data. Minimize privileges. Avoid secret leakage and unsafe logging.        |
| <b>Privacy</b><br>Collect, expose, persist, and retain only necessary data. Mask sensitive fields and respect deletion, consent, residency, and audit requirements.                       | <b>Performance</b><br>Measure or reason about critical paths. Avoid N+1 queries, unbounded processing, blocking I/O, excessive client work, and large payloads.    |
| <b>Accessibility</b><br>Support keyboard use, semantic structure, focus management, readable contrast, labels, announcements, and reduced motion where applicable.                        | <b>Reliability</b><br>Use timeouts, retries with limits, idempotency, graceful degradation, transactional boundaries, health checks, and rollback plans as needed. |
| <b>Observability</b><br>Provide actionable logs, metrics, traces, error context, and correlation IDs without leaking sensitive data.                                                      | <b>Maintainability</b><br>Favor explicit, testable, local reasoning. Document non-obvious decisions. Keep modules cohesive and interfaces narrow.                  |
| <b>Compatibility</b><br>Preserve documented APIs, data formats, migrations, browser/device support, and downstream consumer behavior unless change is intentional.                        | <b>Documentation</b><br>Update the nearest source of truth. Explain new configuration, commands, operational actions, migrations, and behavior changes.            |

## Universal negative instructions

### Guardrail Addendum

Do not:

- invent product requirements not stated or discoverable from authoritative sources;
- rewrite unrelated code for style or personal preference;
- add dependencies, services, flags, or abstraction layers without need;
- weaken validation, authorization, type safety, tests, or error handling;
- silently change public APIs, schemas, permissions, defaults, or data meaning;
- claim a test, build, visual check, benchmark, or deployment succeeded without running it;
- hide a blocker behind a confident completion summary;
- leave mock data, debug output, disabled tests, broad exceptions, or TODO-based correctness in production paths.



*Negative instructions are useful when they describe concrete failure modes, not vague cautions.*

## **Reasonable autonomy policy**

- Proceed without asking when the decision is local, reversible, and consistent with existing conventions.
- State an assumption when information is missing but a safe default exists.
- Ask or stop when the missing decision changes business meaning, security, privacy, irreversible data migration, billing, legal obligations, or public compatibility.
- Prefer a small reversible implementation over a broad speculative platform.

# Domain Packs

Attach one or more specialized modules based on the project type.

Paste the relevant addendum after the Master Task Prompt. These packs focus the agent on domain-specific evidence and quality concerns without bloating every request.

## Web and frontend

### Web and frontend pack

## Web/frontend addendum

- Reuse the existing design system, layout primitives, tokens, and interaction patterns.
- Verify responsive behavior at {{VIEWPORTS}}.
- Preserve semantic HTML, keyboard navigation, focus visibility, labels, and screen-reader feedback.
- Cover loading, empty, error, partial, disabled, optimistic, and success states.
- Avoid generic template aesthetics. Follow {{DESIGN\_DIRECTION}} and use realistic content.
- Inspect the rendered result, console, network requests, layout overflow, and interaction behavior in the browser.
- Do not change global visual language unless explicitly requested.

## Backend and API

### Backend and API pack

## Backend/API addendum

- Treat the published contract and schema as sources of truth.
- Validate at boundaries and return stable, documented error shapes.
- Enforce authorization server-side for every privileged operation.
- Consider transactions, concurrency, idempotency, pagination, rate limits, timeouts, retries, and partial failures.
- Avoid N+1 queries and unbounded reads or writes.
- Add contract, integration, and failure-path tests.
- Verify representative requests and inspect logs/data effects.

## Full-stack product

### Full-stack product pack

## Full-stack addendum

- Trace the user flow from interface through API, domain logic, persistence, asynchronous work, and feedback to the user.
- Keep validation and authorization at trusted server boundaries, while also providing immediate client feedback.
- Keep client and server contracts synchronized.
- Handle eventual consistency, duplicate submissions, retries, stale state, and partial failure.
- Verify the complete flow with representative data, not isolated components.

## Mobile application

### Mobile application pack

## Mobile addendum

- Follow platform conventions for navigation, gestures, permissions, system UI, accessibility, and lifecycle behavior.
- Verify on {{DEVICE\_MATRIX}} and relevant OS versions.
- Handle offline, poor network, backgrounding, interruption, deep links, keyboard appearance, safe areas, and permission denial.
- Minimize startup cost, memory use, battery impact, and unnecessary network work.
- Preserve secure local storage and avoid sensitive data in logs or screenshots.
- Exercise the real flow on emulator/simulator or device.

## Desktop application

### Desktop application pack

## Desktop addendum

- Respect platform window, menu, keyboard, file, notification, and accessibility conventions.
- Handle resizing, multiple windows, high-DPI displays, file permissions, auto-update behavior, and interrupted operations.
- Keep privileged native capabilities behind narrow trusted interfaces.
- Verify installation/build packaging and representative OS behavior.

## CLI and developer tool

### CLI and developer tool pack

## CLI/developer-tool addendum

- Preserve documented flags, exit codes, stdout/stderr behavior, configuration, and machine-readable output.
- Provide useful help, validation, diagnostics, and remediation guidance.
- Support non-interactive and CI use where applicable.
- Avoid destructive defaults; require explicit confirmation or flags.
- Test common commands, invalid input, missing dependencies, pipes, signals, partial writes, and platform-specific paths.

## Library or SDK

### Library or SDK pack

## Library/SDK addendum

- Preserve public API and semantic-versioning expectations.
- Keep the surface minimal, composable, typed, and well documented.
- Add examples and tests that behave like downstream consumers.
- Avoid leaking implementation details into public types or errors.
- Verify packaging, tree-shaking/import behavior, generated artifacts, and compatibility across {{SUPPORTED\_VERSIONS}}.

## Data and analytics

### Data and analytics pack

## Data/analytics addendum

- Define source lineage, grain, time semantics, null handling, deduplication, late arrivals, and quality expectations.
- Make transformations deterministic and idempotent where possible.
- Validate row counts, uniqueness, referential integrity, ranges, and freshness.
- Avoid silently changing metric definitions.
- Include representative samples, backfill strategy, cost/performance checks, and monitoring for pipeline failures or drift.

## Machine learning and AI

### Machine learning and AI pack

## ML/AI addendum

- Define the task, baseline, dataset splits, metrics, error costs, and acceptance thresholds before optimization.
- Prevent data leakage and document feature/data lineage.
- Evaluate representative slices, not only aggregate metrics.
- Track reproducibility: code, data version, model, parameters, and environment.
- Consider latency, cost, safety, privacy, bias, drift, fallback behavior, and human review.
- Do not claim model quality without measured evaluation evidence.

## Infrastructure and DevOps

### Infrastructure and DevOps pack

## Infrastructure/DevOps addendum

- Prefer declarative, reviewable, idempotent changes.
- Preserve environment separation and least privilege.
- Plan migration, rollout, observability, rollback, state handling, and disaster recovery before applying production changes.
- Validate syntax, plans/diffs, policy checks, secrets handling, health checks, quotas, and cost impact.
- Never perform destructive or production-affecting actions without explicit authorization and a reviewed recovery path.

## Security-sensitive system

### Security-sensitive system pack

## Security-sensitive addendum

- Document assets, actors, trust boundaries, threat scenarios, and abuse cases.
- Enforce deny-by-default authorization and least privilege.
- Use established cryptographic and authentication libraries; do not invent security protocols.
- Validate inputs, outputs, redirects, file paths, deserialization, and external calls.
- Avoid sensitive data in logs, analytics, errors, client bundles, caches, and test fixtures.
- Add negative tests and report residual risks explicitly.

## Prototype or research

### Prototype or research pack

## Prototype/research addendum

- State which dimensions are exploratory and which must still be production-safe.
- Optimize for learning speed without faking evidence or hard-coding conclusions.
- Keep experiments reproducible and assumptions visible.
- Separate disposable scaffolding from code intended to survive.
- Define the decision the prototype must inform and the evidence required.

# Task Packs

Reusable prompts for common categories of engineering work.

---

## New feature

### New feature task pack

Implement `{{FEATURE}}` for `{{TARGET_USER}}`.

First trace the current journey and identify the smallest end-to-end slice. Define the user-visible states, business rules, data/API changes, permissions, analytics/observability, and test strategy. Reuse existing patterns. Verify the complete journey with representative data and map every acceptance criterion to evidence. Do not redesign or refactor unrelated areas.

## Bug fix

### Bug fix task pack

Investigate and fix `{{BUG}}`.

Reproduce the issue first and record the evidence. Determine the root cause, not only the visible symptom. Identify the smallest safe fix and likely regression surface. Add a test that fails before the fix and passes after it when feasible. Verify the original reproduction path plus adjacent edge cases. Report the root cause, changed behavior, and evidence.

## Refactor

### Refactor task pack

Refactor `{{AREA}}` to achieve `{{REFACTOR_GOAL}}` without changing observable behavior except `{{INTENTIONAL_CHANGES}}`.

Characterize current behavior and test coverage before editing. Preserve public contracts, data meaning, error behavior, ordering, timing-sensitive semantics, and compatibility. Make incremental changes, run tests after each coherent step, and provide evidence that behavior remains equivalent.

## Migration

### Migration task pack

Plan and implement the migration from `{{CURRENT_SYSTEM}}` to `{{TARGET_SYSTEM}}`.

Include compatibility, data transformation, dual-read/write or phased rollout, backfill, idempotency, validation, monitoring, rollback, and cleanup. Separate schema/application/deployment steps. Protect against partial execution and mixed versions. Do not remove the old path until exit criteria are met.

## Integration

### Integration task pack

Integrate `{{EXTERNAL_SERVICE}}` for `{{USE_CASE}}`.

Use the official contract and existing integration patterns. Define auth, secret management, timeouts, retries, idempotency, rate limits, error mapping, webhook verification, reconciliation, observability, sandbox testing, and fallback behavior. Never expose provider-specific details unnecessarily to product code or users.

## Performance improvement

### Performance improvement task pack

Improve `{{PERFORMANCE_AREA}}` from `{{BASELINE}}` to `{{TARGET}}`.

Measure or reproduce the baseline first. Identify the dominant bottleneck and avoid speculative optimization. Implement the smallest meaningful change, measure again under comparable conditions, check correctness and resource trade-offs, and report method, results, variance, and remaining bottlenecks.

## Security hardening

### Security hardening task pack

Harden `{{AREA}}` against `{{THREATS}}`.

Map assets, actors, trust boundaries, authorization decisions, untrusted inputs, and sensitive outputs. Prefer established platform controls. Add negative tests for abuse cases. Verify that functionality remains usable and report residual risk. Do not rely on client-side checks as security controls.

## Test coverage

### Test coverage task pack

Add high-value tests for `{{AREA}}`.

Prioritize business invariants, public behavior, failure paths, permissions, and regression-prone boundaries. Avoid tests that only mirror implementation or overuse snapshots. Use representative fixtures, deterministic time/randomness, and clear failure messages. Run the targeted suite and relevant broader checks.

## Documentation

### Documentation task pack

Update documentation for `{{CHANGE}}`.

Identify the correct audience and source of truth. Explain prerequisites, behavior, examples, configuration, limitations, failure recovery, and migration steps where relevant. Verify commands and examples against the actual product. Remove or correct stale information nearby only when directly related.

## Code review

### Code review task pack

Review the changes for {{TASK}} as a skeptical senior engineer.

Check correctness, scope, architecture, security, privacy, performance, accessibility, reliability, compatibility, tests, operations, and documentation. Trace at least one happy path and one failure path. Rank findings by severity, reference concrete files/lines, explain impact, and suggest the smallest fix. Do not invent issues unsupported by the diff or repository.

## Release readiness

### Release readiness task pack

Assess release readiness for {{CHANGE}}.

Verify acceptance criteria, automated checks, real-flow behavior, migration and rollback, configuration, feature flags, monitoring, alerts, support impact, documentation, known risks, and ownership. Produce a go/no-go recommendation with explicit evidence and blockers. Do not treat “build passes” as sufficient.

## Incident remediation

### Incident remediation task pack

Remediate {{INCIDENT}} safely.

Preserve evidence and prioritize containment. Separate immediate mitigation, root-cause correction, recovery validation, and long-term prevention. Consider customer impact, data integrity, security, observability, communication, and rollback. Add regression detection and document follow-up owners. Avoid broad changes during active mitigation unless necessary.

# Verification and Evidence

Turn “done” into a falsifiable claim.

The final answer should show how the agent knows the change works. Evidence depends on the project, but every acceptance criterion should have an observable proof source.

## Evidence ladder

1. Static evidence: types, lint, formatting, schema validation, policy checks.
2. Automated behavior evidence: unit, integration, contract, end-to-end, property, or migration tests.
3. Runtime evidence: real commands, requests, device/browser interactions, job runs, or sample outputs.
4. State evidence: database changes, files, events, queues, logs, metrics, traces, or screenshots.
5. Review evidence: focused diff inspection, acceptance mapping, edge-case analysis, and known-risk disclosure.

## Universal verification matrix

| Placeholder          | What to enter                                                                         | Example            |
|----------------------|---------------------------------------------------------------------------------------|--------------------|
| Acceptance criteria  | Each criterion is tied to a test, runtime observation, or explicit unverified status. | {{EVIDENCE_OR_NA}} |
| Static quality       | Typecheck/lint/schema/policy commands pass where available.                           | {{EVIDENCE_OR_NA}} |
| Automated tests      | Relevant targeted tests pass; broader suite is run when risk warrants.                | {{EVIDENCE_OR_NA}} |
| Build/package        | Production build, package, image, or artifact creation succeeds.                      | {{EVIDENCE_OR_NA}} |
| Real flow            | The affected path is exercised in the running system.                                 | {{EVIDENCE_OR_NA}} |
| Failure behavior     | At least one representative invalid or failing path is observed.                      | {{EVIDENCE_OR_NA}} |
| Permissions/security | Role and trust-boundary behavior is checked.                                          | {{EVIDENCE_OR_NA}} |
| Data/state           | Writes, migrations, events, or outputs are inspected for correctness.                 | {{EVIDENCE_OR_NA}} |
| UX/accessibility     | Rendered behavior, keyboard/focus, responsiveness, and feedback are inspected.        | {{EVIDENCE_OR_NA}} |



| Placeholder         | What to enter                                                                            | Example            |
|---------------------|------------------------------------------------------------------------------------------|--------------------|
| <b>Performance</b>  | Target or regression risk is measured or explicitly assessed.                            | {{EVIDENCE_OR_NA}} |
| <b>Operations</b>   | Logs, metrics, rollout, configuration, and rollback implications are checked.            | {{EVIDENCE_OR_NA}} |
| <b>Diff hygiene</b> | No accidental files, secrets, debug output, unrelated formatting, or scope creep remain. | {{EVIDENCE_OR_NA}} |

### Acceptance-to-evidence table request

Before the final report, produce this table:

```
| Acceptance criterion | Evidence source | Result | Notes |
|---|---|---|---|
| {{CRITERION_1}} | {{TEST_OR_OBSERVATION}} | PASS/FAIL/UNVERIFIED | {{NOTES}} |
```

Do not mark PASS based only on code inspection when runtime or test evidence is available. Explain every UNVERIFIED item and its risk.

*This small table sharply reduces confident but unsupported completion claims.*

## Final report template

### Evidence-Based Completion Report

```
## Outcome
{{WHAT_CHANGED_AND_USER_IMPACT}}

## Changed files
- {{FILE}} - {{WHY}}

## Verification performed
- {{COMMAND_OR_ACTION}} - {{RESULT}}

## Acceptance evidence
{{ACCEPTANCE_TO_EVIDENCE_SUMMARY}}

## Assumptions and decisions
- {{ASSUMPTION_OR_DECISION}}

## Remaining risk
- {{RISK_OR_NONE}}

## Not performed
- {{UNVERIFIED_CHECK_AND_REASON_OR_NONE}}

"Not performed" is a trust feature, not an embarrassment.
```

# Recovery and Debugging

Prompts for when the agent is stuck, looping, or producing generic work.

## When the output is generic

### Specificity Recovery

Pause implementation. Compare the current result with:

- {{PRODUCT\_PRINCIPLES}}
- {{DESIGN\_OR\_ARCHITECTURE\_EXAMPLES}}
- {{EXISTING\_STRONG\_REFERENCE}}
- {{ACCEPTANCE\_CRITERIA}}

List the five most generic or project-inconsistent decisions. For each, explain the concrete replacement and source of truth. Then revise only those decisions and verify the affected flow.

## When the agent changes too much

### Scope Recovery

Review the diff against {{TASK}} and {{NON\_GOALS}}.

Classify every changed file as required, supporting, incidental, or unrelated. Revert incidental and unrelated changes unless they are necessary for correctness and explicitly justified. Preserve user-authored changes. Return a focused diff summary before continuing verification.

## When tests fail repeatedly

### Failure Triage

Stop making speculative edits.

1. Reproduce the earliest failing check in isolation.
2. Capture the exact error and relevant state.
3. Identify whether the failure is product code, test code, environment, dependency, fixture, or pre-existing behavior.
4. Trace the causal path and propose one falsifiable hypothesis.
5. Make the smallest experiment or fix to test that hypothesis.
6. Re-run the isolated check before touching other areas.
7. Report evidence if blocked.

## When the implementation is visually weak

### Visual Quality Recovery

Inspect the rendered result at {{VIEWPORTS}} and compare it with {{DESIGN\_DIRECTION}} and {{REFERENCE\_SCREENSHOTS}}.

Evaluate hierarchy, spacing rhythm, typography, alignment, density, content, interaction feedback, empty/error states, and responsive behavior. Identify the

highest-impact three defects, correct them using existing design tokens and components, then re-inspect the actual rendered flow. Avoid global redesign.

## When context is too large

### Context Compression

Create a task-specific context packet containing only:

- objective and acceptance criteria;
- permanent project rules;
- relevant file map and architecture summary;
- public contracts and business invariants;
- current behavior and reproduction evidence;
- constraints, risks, and required checks.

Omit unrelated history, full files that can be read directly, duplicate rules, and speculative discussion. Mark authoritative sources explicitly.

## When the agent says it is done too early

### Completion Challenge

Assume the implementation is not complete. Attempt to disprove completion by:

- mapping every acceptance criterion to evidence;
- exercising one invalid, one permission-related, and one failure scenario;
- checking the final diff for hidden scope or incomplete states;
- inspecting runtime logs/output;
- checking the relevant quality guardrails.

Fix defects found. Clearly list anything that remains unverified.

### Best debugging rule

Ask for a smaller falsifiable experiment, not a larger explanation.

# Quality Scorecard

Review results consistently and convert defects into better rules.

Score each dimension from 0 to 5. A “5” requires evidence. A polished interface with weak authorization or untested migrations is not high quality overall.

| Dimension                     | What excellent looks like                                                                           | Score       | Evidence     |
|-------------------------------|-----------------------------------------------------------------------------------------------------|-------------|--------------|
| <b>Outcome fit</b>            | The implemented behavior directly solves the stated user/system outcome.                            | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Correctness</b>            | Business invariants, edge cases, and failure paths are handled and tested.                          | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Scope discipline</b>       | The diff is focused and preserves unrelated behavior.                                               | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Architecture</b>           | The design fits existing boundaries and remains understandable.                                     | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>UX/product quality</b>     | Flows, feedback, states, content, and visual hierarchy are coherent.                                | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Accessibility</b>          | Relevant semantic, keyboard, focus, label, contrast, and assistive behavior is verified.            | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Security/privacy</b>       | Trust boundaries, permissions, validation, data exposure, and secret handling are sound.            | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Performance</b>            | Critical-path cost and resource use meet targets or are measured responsibly.                       | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Reliability/operations</b> | Failures, retries, rollout, rollback, logs, metrics, and support needs are considered.              | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Compatibility</b>          | Public contracts, schemas, consumers, platforms, and data meaning are preserved or migrated safely. | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Testing/evidence</b>       | The result is supported by relevant automated and runtime evidence.                                 | 0 1 2 3 4 5 | {{EVIDENCE}} |
| <b>Documentation</b>          | Sources of truth and operational/user guidance are current.                                         | 0 1 2 3 4 5 | {{EVIDENCE}} |

## Scoring guide

- **0 - Missing.** The dimension was ignored or contradicted.
- **1 - Fragile.** Major gaps exist; evidence is absent.
- **2 - Partial.** Basic path works but significant conditions remain untreated.
- **3 - Solid.** Expected behavior is covered with reasonable evidence.

- **4 - Strong.** Edge cases and integration concerns are handled well.
- **5 - Exemplary.** Clear evidence, thoughtful trade-offs, and durable prevention of regressions.

## Release threshold template

### Quality Gate Policy

Minimum overall average: {{MIN\_AVERAGE\_SCORE}} / 5  
No dimension below: {{MIN\_DIMENSION\_SCORE}} / 5  
Mandatory dimensions at 4 or higher: {{MANDATORY\_DIMENSIONS}}  
Required evidence: {{REQUIRED\_EVIDENCE}}  
Exception approver: {{EXCEPTION\_OWNER}}  
Exception expiry/remediation date: {{EXCEPTION\_DEADLINE}}

*Averages should not hide a critical zero in security, correctness, or operations.*

## Learning loop

1. Record the defect or review finding.
2. Identify whether the cause was missing context, weak rules, tool limitations, environment problems, or execution failure.
3. Add the smallest durable prevention: instruction, test, lint rule, template, fixture, observability, or documentation.
4. Remove redundant prompt text after the prevention becomes automated.
5. Review recurring failure patterns monthly or after major releases.

# Ready-to-Paste Complete Kit

A single consolidated prompt for a new or weakly configured repository.

This combined template is intentionally comprehensive. Use it to bootstrap a repository, then move the stable sections into project files and shorten future requests.

## Complete Universal Agent Prompt

You are the implementation agent for `{{PROJECT_NAME}}`, a `{{PROJECT_TYPE}}` used by `{{TARGET_USER}}` to achieve `{{USER_OUTCOME}}`.

# Project principles  
`{{PRODUCT_PRINCIPLES}}`

# Technical context

- Stack: `{{TECH_STACK}}`
- Architecture: `{{ARCHITECTURE}}`
- Sources of truth: `{{SOURCE_OF_TRUTH}}`
- Existing conventions: `{{CONVENTIONS}}`
- Commands: `{{PROJECT_COMMANDS}}`

# Task  
`{{TASK}}`

## Current behavior  
`{{CURRENT_BEHAVIOR}}`

## Desired behavior  
`{{DESIRED_BEHAVIOR}}`

## Acceptance criteria  
`{{ACCEPTANCE_CRITERIA}}`

## Non-goals  
`{{NON_GOALS}}`

## Constraints

- Compatibility: `{{COMPATIBILITY_REQUIREMENTS}}`
- Security/privacy: `{{SECURITY_REQUIREMENTS}}`
- Performance: `{{PERFORMANCE_TARGET}}`
- Dependencies: `{{DEPENDENCY_POLICY}}`
- Scope: preserve unrelated behavior and user-authored changes.

# Operating procedure

1. Read relevant instructions and authoritative documentation.
2. Explore the current flow, relevant files, tests, types, schema, APIs, and similar implementations before editing.
3. Summarize observed current behavior, change surface, assumptions, risks, and a concise plan.
4. Implement the smallest coherent end-to-end solution using existing patterns.
5. Keep business rules testable and enforce validation/authorization at trusted boundaries.
6. Handle relevant loading, empty, error, partial, permission, timeout, retry, and success states.
7. Add or update high-value tests.
8. Run the relevant static checks, tests, production build, and real runtime flow.
9. Inspect logs, outputs, state changes, network/API behavior, and rendered UI or device behavior as applicable.
10. Test representative edge cases and at least one failure path.
11. Review the final diff for accidental scope, secrets, debug artifacts, dead code, disabled tests, and unrelated formatting.

12. Fix defects found and repeat the affected verification.

# Quality bar

- Correctness: preserve invariants and handle boundaries and failures.
- Product/UX: maintain coherent flows, feedback, responsiveness, accessibility, and the design direction {{DESIGN\_DIRECTION}}.
- Architecture: respect existing boundaries and avoid unnecessary abstractions.
- Security/privacy: use least privilege, validate untrusted input, protect sensitive data, and do not rely on client checks for authorization.
- Performance: avoid unnecessary work, unbounded operations, repeated queries, blocking paths, and oversized payloads.
- Reliability/operations: consider idempotency, timeouts, retries, transactions, rollout, rollback, logs, metrics, and support impact where relevant.
- Compatibility: do not silently change public APIs, data meaning, schemas, configuration, or supported platforms.

# Prohibited behavior

Do not invent material product decisions, add unnecessary dependencies, perform unrelated refactors, weaken tests or validation, expose secrets, or claim a check passed without running it. Do not hide blockers behind a completion claim.

# Required verification

{{TEST\_COMMANDS}}  
{{RUNTIME\_VERIFICATION}}  
{{EDGE\_CASES\_TO\_VERIFY}}

# Final response

Provide:

1. Outcome and user/system impact.
2. Changed files and purpose.
3. Acceptance criteria mapped to evidence.
4. Checks actually run and results.
5. Assumptions and decisions.
6. Remaining risks and anything not verified.

A task is not complete until the acceptance criteria are supported by evidence.

## Recommended context attachments

- [ ] Project Profile or product specification.
- [ ] Relevant instruction file(s).
- [ ] Design references or representative screenshots for visual work.
- [ ] API contract, schema, migration policy, or event definitions for data work.
- [ ] Reproduction steps, logs, traces, and example inputs for bugs.
- [ ] Performance baseline and measurement method for optimization.
- [ ] Threat model or trust-boundary notes for security-sensitive work.
- [ ] Release, deployment, rollback, and monitoring details for production changes.

# Worked Example

A filled prompt for a full-stack product feature.

This example demonstrates the level of specificity to aim for. It is not a universal architecture recommendation.

## Example - Bulk case assignment

You are working in Northstar Console, a B2B support operations web app used by support team leads to resolve escalated customer cases quickly.

### ## Objective

Add bulk assignment to the existing case table.

### ## Current behavior

A team lead must open each case and select an assignee individually.

### ## Desired behavior

A lead can select multiple eligible cases, choose one active teammate, review how many cases will be changed, and confirm a single bulk operation. The table updates without a full page reload and clearly reports partial failures.

### ## Acceptance criteria

- Eligible visible cases can be selected individually or with "select page."
- Cases outside the lead's team, closed cases, and cases already locked by an active workflow are disabled with an accessible explanation.
- The action bar shows selected count and provides Clear and Assign actions.
- The assignee list contains active members of the current team only.
- Confirmation shows count, assignee, and irreversible side effects.
- The server re-checks every permission and eligibility rule transactionally.
- Duplicate submissions are idempotent for five minutes.
- Partial failure returns per-case results; successful rows update immediately.
- An audit event records actor, target cases, assignee, result, and correlation ID.
- Keyboard navigation, focus, mobile overflow, loading, error, and success states are verified.

### ## Non-goals

- No redesign of the table.
- No cross-team assignment.
- No new state-management dependency.
- No bulk reassignment history screen.

### ## Sources of truth

- Product behavior: docs/case-assignment.md
- Permissions: src/domain/cases/permissions.ts
- Existing single assignment: src/features/cases/AssignCaseDialog.tsx
- API patterns: src/server/api/cases/
- Schema: prisma/schema.prisma

### ## Constraints

- Next.js, TypeScript, tRPC, Prisma, Postgres.
- Preserve the existing single-case assignment flow.
- Use existing Dialog, Combobox, Toast, and table-selection primitives.
- No schema migration unless audit metadata cannot be represented currently.
- P95 API target under 500 ms for 100 selected cases.
- Tenant isolation and team authorization are mandatory server-side.

### ## Workflow

First trace the current table selection model, single assignment flow, permission rules, audit event system, and transaction patterns. Provide a concise plan with risks. Implement the smallest complete end-to-end change, add tests, run the checks, exercise the real browser flow with representative eligible and



ineligible cases, inspect network responses and database/audit effects, then review the final diff.

#### ## Verification

- pnpm typecheck
- pnpm lint
- pnpm test -- cases
- pnpm test:integration -- bulk-assignment
- pnpm build
- Browser checks at 390x844, 768x1024, and 1440x900
- Verify success, mixed eligibility, lost permission, duplicate submit, server timeout, and partial failure.

#### ## Completion report

Map each criterion to evidence and list anything not verified. Do not claim completion based only on the passing build.

*Notice that product behavior, server invariants, UX states, operations, and evidence are specified together.*

## Why this prompt works

- It describes a real user and a concrete outcome.
- The existing implementation and sources of truth are named.
- Acceptance criteria cover both interface and server behavior.
- Non-goals prevent a table redesign and unnecessary infrastructure.
- Security and performance requirements are testable.
- Verification uses the running system and representative failure paths.
- The agent has autonomy on local details but not on business meaning.

# One-Page Checklists

Print, pin, or paste these into issue templates.

## Before sending a task

- ☐ The user/system outcome is clear.
- ☐ Current and desired behavior are stated.
- ☐ Acceptance criteria are observable.
- ☐ Non-goals prevent predictable scope creep.
- ☐ Relevant sources of truth are named.
- ☐ Compatibility, security, privacy, and performance constraints are included.
- ☐ The requested verification is possible in the environment.
- ☐ The task is small enough to implement and verify coherently.

## Before accepting completion

- ☐ Every acceptance criterion has evidence.
- ☐ Relevant static checks, tests, and build were actually run.
- ☐ The real affected flow was exercised.
- ☐ At least one failure or invalid path was checked.
- ☐ Permissions and trust boundaries were checked where relevant.
- ☐ Rendered UI/device behavior was inspected where relevant.
- ☐ Data, logs, events, outputs, or migrations were inspected where relevant.
- ☐ The final diff is focused and contains no secrets or debug artifacts.
- ☐ Unverified items and residual risks are explicit.

## When quality is disappointing

- ☐ Was the product outcome concrete?
- ☐ Did the agent inspect authoritative sources?
- ☐ Were the design/architecture defaults explicit?
- ☐ Did it separate exploration from implementation?
- ☐ Could it run and observe the product?
- ☐ Were acceptance criteria tied to evidence?
- ☐ Was the task too broad?
- ☐ Should the recurring mistake become a permanent rule or test?

### The simplest durable formula

Good context + focused task + repository exploration + explicit constraints + real verification + a learning loop.

## Blank task card

### Copy-ready task card

PROJECT: {{PROJECT\_NAME}}  
TASK: {{TASK}}  
USER/OUTCOME: {{TARGET\_USER}} -> {{USER\_OUTCOME}}  
CURRENT: {{CURRENT\_BEHAVIOR}}  
DESIRED: {{DESIRED\_BEHAVIOR}}  
ACCEPTANCE: {{ACCEPTANCE\_CRITERIA}}  
NON-GOALS: {{NON\_GOALS}}  
SOURCES: {{SOURCE\_OF\_TRUTH}}  
CONSTRAINTS: {{CONSTRAINTS}}  
CHECKS: {{TEST\_COMMANDS}}  
RUNTIME PROOF: {{RUNTIME\_VERIFICATION}}  
RISKS: {{KNOWN\_RISKS}}

*End of playbook. Replace the placeholders and start with a focused, observable change.*

# BUILD THE SYSTEM, NOT JUST THE PROMPT.

Context. Constraints. Workflow. Verification. Learning.

This playbook is an original, vendor-neutral workflow template inspired by common agentic software engineering practices. “Lovable” is referenced descriptively as a quality benchmark. This document is not affiliated with, endorsed by, or a reproduction of private prompts from Lovable or OpenAI.

Adapt these templates to your organization's security, privacy, legal, and operational policies.

**Edition 1.0 | August 2026**